

Principles for Creating Testable Code

SOLID Principles

- S – Single Responsibility Principle
- O – Open/Close Principle
- L – Liskov Substitution Principle
- I – Interface Segregation Principle
- D – Dependency Inversion Principle

Single Responsibility Principle

Single Responsibility Principle is where an Object should do one thing and one thing only. For example in a function or component may load some data apply some filtering to the data before loading some more data and then display the data. This function is performing multiple actions and this will make it more difficult to test. It is better (for testability) if each action is performed on its own. (load data, apply filter, load data, display in UI) This means that you will have three separate functions, which can all be tested separately.

Break you code up into small components with each component doing just one thing.

Open/Close Principle is where a class is Open to extension, but Closed to changed.

If new functionality is required of a class it is better to create new methods then it is to update/change existing methods. Data returned from an existing method can be passed to the new method to alter it as required.

A trivial example of this might be: Data returned of a number set is in an ascending order, but it is now required in a descending order. Rather than change the original method we could create a new method that takes in a numbered list and outputs it in a descending order. We now have two functions that can both be tested and remain working after a change.

Liskov Substitution Principle

Liskov Substitution Principle is where a class that uses another class, should be able to accept using another class without it knowing the difference. There should be loose coupling of code. A bit like a magical black box, you feed it a known input and get a known output. You are not concerned with how it did it.

This is an important principle in testability as it allows us to mock external services and easily provide the range of data outputs from the service to properly test the application

Interface Segregation Principle

- No client should be forced to depend on methods it does not use.
- Having a class implement an interface and all of its methods when some of those methods are not required adds complexity when trying to mock a service. Keep your services small by splitting them into multiple, independent parts.

Dependency Inversion Principle

- High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.
- Depend on interfaces/contracts, not concrete implementations